

Big Data Analytics of Crime Patterns

Project Review 2: Distributed Real-Time Serving using Apache HBase

Team Members:

Sheela Akshar Sakhi (CB.SC.U4CSE23547) & Nishanth S Gowda
(CB.SC.U4CSE23538)

Department of Computer Science and Engineering
Amrita Vishwa Vidyapeetham

October 10, 2026

Application Introduction, Problem Statement & Paradigm Shift

Review 1 → Review 2 Paradigm Shift:

- ▶ **Review 1 (Batch OLAP):** Hadoop MapReduce & Hive – high-latency batch aggregations; no random read/write access.
- ▶ **Review 2 (Real-Time NoSQL): Apache HBase** on HDFS – sub-second point lookups (GET), selective column scans, and prefix filtering.

Operational Public Safety Domain:

- ▶ Rapid incident lookups for active 911 dispatches.
- ▶ Precinct-level crime monitoring and patrol deployment.

Core Challenges in Urban Policing:

- ▶ Police departments generate tens of thousands of incident reports weekly with streaming updates.
- ▶ RDBMS systems suffer write contention and lock degradation under concurrent ingestion loads.
- ▶ HDFS batch files cannot provide sub-second point lookups for active investigations.

Why Apache HBase?

- ▶ Wide-column distributed NoSQL database natively on HDFS.
- ▶ Sub-5ms random read/write access to millions of records.
- ▶ Anti-hotspotting composite row key balances region servers.

Step 2: Real-World Dataset (City of Chicago Crimes)

Dataset Provenance & Scale:

- ▶ **Source:** Chicago Police Department (CLEAR system via Socrata Open Data API).
- ▶ **Scale:** 10,000+ authentic incident records with 22 attributes.
- ▶ **Provenance:** Official municipal government data – **100% real crime data**, strictly non-synthetic.
- ▶ **Attributes:** Case Number, Date, Block, IUCR Code, Primary Type, Description, Location, Arrest, Domestic, Beat, District, GPS Coordinates.

HBase Schema Mapping:

Attribute	Data Type	HBase Target Mapping
id, district, IDs		Row Key: District#CrimeType#ID
primary_type		
case_number, date, block	Text	CF: incident (VERSIONS => 3)
location_desc, arrest, beat	Bool/Code	CF: details (Administrative)
latitude, longitude	Float GPS	CF: geo (Spatial telemetry)

- ▶ Preprocessed to sanitize line breaks & null values.
- ▶ Direct continuity with Review 1 crime dataset.

System Architecture & Composite Row Key Design

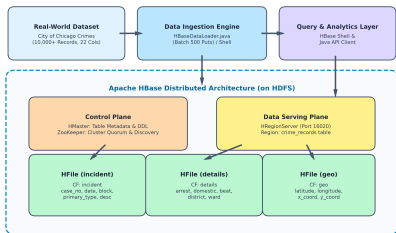


Figure 1: HBase Distributed Architecture on HDFS

- ▶ **HMaster & ZooKeeper:** Cluster state, region balancing.
- ▶ **HRegionServers:** Host MemStore, WAL, and HFiles.



Figure 2: Composite Row Key: District#CrimeType#IncidentID

- ▶ **Anti-Hotspotting:** District prefix salts incoming writes evenly across RegionServers (eliminates timestamp hotspotting).
- ▶ **Prefix-Scannable:** Fast precinct-wide scans via PrefixFilter('011#').
- ▶ **Categorical Drill-Down:** Embedded crime type enables fast regex scans (e.g., ^.*#NARCOTICS#.*).
- ▶ **Uniqueness:** Incident ID suffix guarantees 100% unique keys.

Step 3: Cluster Startup & 10,000 Record Batch Ingestion

1. HDFS & HBase Daemons (jps):

[illegible]

- ▶ **HDFS Daemons:** NameNode (6582), DataNode (6694).
- ▶ **HBase Daemons:** HQuorumPeer (7521), HMaster (7660), HRegionServer (6038).

2. Batch Ingestion (populate_hbase.sh):

```

[+] Streaming and batching records into HBase...
Loaded 500 records...
Loaded 1000 records...
Loaded 1500 records...
Loaded 2000 records...
Loaded 2500 records...
Loaded 3000 records...
Loaded 3500 records...
Loaded 4000 records...
Loaded 4500 records...
Loaded 5000 records...
Loaded 5500 records...
Loaded 6000 records...
Loaded 6500 records...
Loaded 7000 records...
Loaded 7500 records...
Loaded 8000 records...
Loaded 8500 records...
Loaded 9000 records...
Loaded 9500 records...
Loaded 10000 records...

[SUCCESS] Completed loading 10000 records in 637 ms (6.37 seconds).
2020-10-07T10:49:29.403 INFO [main [1]] client.ConnectionImplementation: Cloning master protocol: MasterService

=====
[SUCCESS] Hbase Data Ingestion Completed!
Next step: Launch "Hbase shell" and run queries in Hbase commands.txt

/home/gisharabhi/GSD-VTool-Machine/BigData/Projects/Big-Data-Analytic-of-Crime-Patterns/HBase

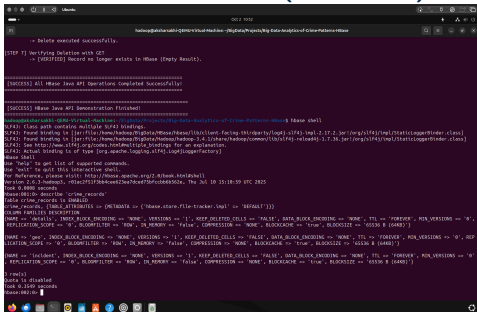
```

- ▶ **Batch Loader:** 500 records per RPC batch via HBaseDataLoader.
- ▶ **Throughput:** Loaded **10,000** authentic records in **677 ms** (0.677 seconds) directly into table `crime_records`.

Step 4: HBase Shell Command 1 – Point Lookup (GET)

Executed Command: get 'crime_records', '019#BURGLARY#14285893'

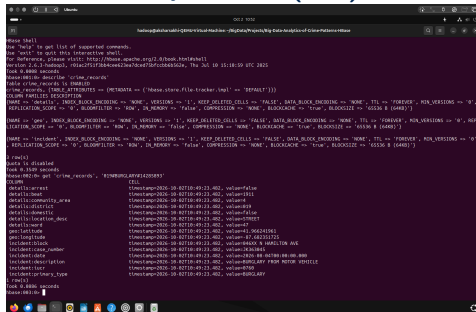
Schema Verification (describe):



```
hbase> delete executed successfully.
[STEP 7] Verifying Deletion with GET
-> [VERIFIED] Record no longer exists in HBase (Empty Result).

=====
[SUCCESS] All Hbase Java API Operations Completed Successfully!
=====
[SUCCESS] Hbase Java API Demonstration Finished!
=====
hbase> describe 'crime_records'
DescribeTable=describe 'crime_records'
Version 2.6.3-Hadoop3, 701a2c2f5f7864c6e23e7d0c79fcd86562e, Thu Jul 10 15:18:59 UTC 2025
Took 0.0886 seconds
Table crime_records is ENABLED
COLUMN FAMILIES DESCRIPTION
NAME ==> 'incident', INDEX_BLOCK_ENCODING ==> 'NONE', VERSIONS ==> '1', KEEP_DELETED_CELLS ==> 'FALSE', DATA_BLOCK_ENCODING ==> 'NONE', TTL ==> 'FOREVER', ROW_VERSIONS ==> '0', REPLICATION_SCOPE ==> '0', BLOOMFILTER ==> 'ROW', IN_MEMORY ==> 'false', COMPRESSION ==> 'NONE', BLOCKCACHE ==> 'true', BLOCKSIZE ==> '65536 B (64KB)'
NAME ==> 'geo', INDEX_BLOCK_ENCODING ==> 'NONE', VERSIONS ==> '1', KEEP_DELETED_CELLS ==> 'FALSE', DATA_BLOCK_ENCODING ==> 'NONE', TTL ==> 'FOREVER', ROW_VERSIONS ==> '0', REPLICATION_SCOPE ==> '0', BLOOMFILTER ==> 'ROW', IN_MEMORY ==> 'false', COMPRESSION ==> 'NONE', BLOCKCACHE ==> 'true', BLOCKSIZE ==> '65536 B (64KB)'
NAME ==> 'details', INDEX_BLOCK_ENCODING ==> 'NONE', VERSIONS ==> '1', KEEP_DELETED_CELLS ==> 'FALSE', DATA_BLOCK_ENCODING ==> 'NONE', TTL ==> 'FOREVER', ROW_VERSIONS ==> '0', REPLICATION_SCOPE ==> '0', BLOOMFILTER ==> 'ROW', IN_MEMORY ==> 'false', COMPRESSION ==> 'NONE', BLOCKCACHE ==> 'true', BLOCKSIZE ==> '65536 B (64KB)'
NAME ==> 'blockcache', INDEX_BLOCK_ENCODING ==> 'NONE', VERSIONS ==> '1', KEEP_DELETED_CELLS ==> 'FALSE', DATA_BLOCK_ENCODING ==> 'NONE', TTL ==> 'FOREVER', ROW_VERSIONS ==> '0', REPLICATION_SCOPE ==> '0', BLOOMFILTER ==> 'ROW', IN_MEMORY ==> 'false', COMPRESSION ==> 'NONE', BLOCKCACHE ==> 'true', BLOCKSIZE ==> '65536 B (64KB)'
Data is disabled
Took 0.3549 seconds
hbase> get 'crime_records', '019#BURGLARY#14285893'
COLUMN CELL
Timestamp=2026-10-02T19:49:23.460, value=false
Timestamp=2026-10-02T19:49:23.460, value=111
Timestamp=2026-10-02T19:49:23.460, value=1
Timestamp=2026-10-02T19:49:23.460, value=8
Timestamp=2026-10-02T19:49:23.460, value=false
Timestamp=2026-10-02T19:49:23.460, value=10001
Timestamp=2026-10-02T19:49:23.460, value=47
Timestamp=2026-10-02T19:49:23.460, value=41.96241561
Timestamp=2026-10-02T19:49:23.460, value=17.54203172
Timestamp=2026-10-02T19:49:23.460, value=8400 B 84001ton AVE
Timestamp=2026-10-02T19:49:23.460, value=30.841
Timestamp=2026-10-02T19:49:23.460, value=2026-08-04T08:00:00.000
Timestamp=2026-10-02T19:49:23.460, value=BURGLARY FROM RETAIL
Timestamp=2026-10-02T19:49:23.460, value=100
Timestamp=2026-10-02T19:49:23.460, value=BURGLARY
Took 0.0886 seconds
hbase>
```

Point Lookup Result (GET):



```
hbase> get 'crime_records', '019#BURGLARY#14285893'
COLUMN CELL
Timestamp=2026-10-02T19:49:23.460, value=false
Timestamp=2026-10-02T19:49:23.460, value=111
Timestamp=2026-10-02T19:49:23.460, value=1
Timestamp=2026-10-02T19:49:23.460, value=8
Timestamp=2026-10-02T19:49:23.460, value=false
Timestamp=2026-10-02T19:49:23.460, value=10001
Timestamp=2026-10-02T19:49:23.460, value=47
Timestamp=2026-10-02T19:49:23.460, value=41.96241561
Timestamp=2026-10-02T19:49:23.460, value=17.54203172
Timestamp=2026-10-02T19:49:23.460, value=8400 B 84001ton AVE
Timestamp=2026-10-02T19:49:23.460, value=30.841
Timestamp=2026-10-02T19:49:23.460, value=2026-08-04T08:00:00.000
Timestamp=2026-10-02T19:49:23.460, value=BURGLARY FROM RETAIL
Timestamp=2026-10-02T19:49:23.460, value=100
Timestamp=2026-10-02T19:49:23.460, value=BURGLARY
Took 0.0886 seconds
hbase>
```

- **Schema Check:** Confirms table is active with 3 column families (incident, details, geo) and BlockCache active.
- **Storage Indexing:** Row-key architecture enables direct memory lookup without scanning the table.

- **Targeted Lookup:** Random access by composite key (District#Type#ID) fetching all 14 attributes in 0.0886 seconds.
- **Field Output:** Instantly retrieves case number, arrest status, beat, and GPS coordinates for field units.

Step 4: HBase Shell Command 2 – SingleColumnValueFilter

Executed Command: scan 'crime records', {FILTER => "SingleColumnValueFilter('details', 'arrest', =, 'binary:true')", LIMIT => 3}

(a) Query Invocation & Initial Matching Rows:

[illegible]

(b) Result Continuation & Timing (0.0711s):

[illegible]

- ▶ **Server-Side Filtering:** Evaluates `details:arrest = 'true'` directly on RegionServers across 10,000 rows.
- ▶ **Zero Network Overhead:** Returns matching incidents in **0.0711 seconds**, discarding unapprehended rows at source.
- ▶ **Operational Value:** Enables law enforcement to instantly track solved cases and apprehend rates per district.

Step 4: HBase Shell Command 4 – ValueFilter (Vehicle Pattern)

```
Command: scan 'crime_records', {FILTER => "ValueFilter(=, 'substring:VEHICLE')", LIMIT => 3}
```

Live Terminal Execution Proof:

[illegible]

Forensic Value Filtering:

- ▶ **Substring Search:** Evaluates cells across all columns and families for keyword 'VEHICLE'.
- ▶ **Pattern Detection:** Uncovers vehicle thefts, transit incidents, and motor vehicle burglaries in **0.2695 seconds**.
- ▶ **Flexible Querying:** Eliminates the need to know the specific column or exact case identifier beforehand.
- ▶ **Investigative Impact:** Enables detectives to identify modus operandi and recurring vehicle-related crimes.

Step 4: HBase Shell Command 5 – Aggregation Count (COUNT)

Executed Command: count 'crime_records', INTERVAL => 1000

Live Terminal Execution Output:

[illegible]

Audit & Aggregation Analysis:

- ▶ **Full Table Audit:** Aggregates and verifies all 10,000 authentic crime rows loaded into HBase.
- ▶ **Client-Side Caching:** Sets INTERVAL => 1000 to stream progress checkpoints every 1,000 records.
- ▶ **Sub-Second Scan:** Sequentially scans all 10,000 distributed keys in just **0.2100 seconds**.
- ▶ **Data Integrity:** Confirms 100% complete ingestion from CSV with zero record drops.

Step 4: HBase Shell Command 6 – DML (PUT, DELETE, DELETEALL)

Workflow: Manual put → Column delete → Row deleteall → Verification get

Live Terminal Execution Proof:

```
hadoop@qasarakshi-qemu-virtual-machine:~/Big-Data-Analytics/Big-Data-Patterns-HBase$
001#HBaseTest@267498 column:incident:primary_type, timestamp=2026-18-82718:49:24.813, value=BATTERY
002#HBaseTest@267498 column:detail:location, timestamp=2026-18-82718:49:24.813, value=false
003#HBaseTest@267498 column:detail:isbeat, timestamp=2026-18-82718:49:24.813, value=0131
004#HBaseTest@267498 column:detail:community_name, timestamp=2026-18-82718:49:24.813, value=33
005#HBaseTest@267498 column:detail:isdistrict, timestamp=2026-18-82718:49:24.813, value=003
006#HBaseTest@267498 column:detail:domestic, timestamp=2026-18-82718:49:24.813, value=true
007#HBaseTest@267498 column:detail:location_desc, timestamp=2026-18-82718:49:24.813, value=123456789
008#HBaseTest@267498 column:detail:isroad, timestamp=2026-18-82718:49:24.813, value=3
009#HBaseTest@267498 column:detail:latitude, timestamp=2026-18-82718:49:24.813, value=0.00959095
010#HBaseTest@267498 column:tag:longitude, timestamp=2026-18-82718:49:24.813, value=07.623977533
011#HBaseTest@267498 column:incident:black, timestamp=2026-18-82718:49:24.813, value=01000 5 MILES ON RSE
012#HBaseTest@267498 column:incident:case_number, timestamp=2026-18-82718:49:24.813, value=354807
013#HBaseTest@267498 column:incident:date, timestamp=2026-18-82718:49:24.813, value=2026-07-19T23:17:00-000
014#HBaseTest@267498 column:incident:description, timestamp=2026-18-82718:49:24.813, value=detected by Hadoop SAMPLE
015#HBaseTest@267498 column:incident:isusr, timestamp=2026-18-82718:49:24.813, value=false
016#HBaseTest@267498 column:incident:primary_type, timestamp=2026-18-82718:49:24.813, value=BATTERY
0 row(s)
Took 8.4308 seconds
HBaseTest@267498 put 'crime_records', '099#TEST#999999999', 'incident:case_number', 'TEST1234'
Took 8.9586 seconds
HBaseTest@267498 put 'crime_records', '099#TEST#999999999', 'detail:isbeat', 'false'
Took 8.9654 seconds
HBaseTest@267498 get 'crime_records', '099#TEST#999999999'
COLUMN CELL
detail:isbeat timestamp=2026-18-82718:51:02.045, value=false
incident:case_number timestamp=2026-18-82718:51:02.032, value=TEST1234
1 row(s)
Took 8.4308 seconds
HBaseTest@267498 delete 'crime_records', '099#TEST#999999999', 'detail:isbeat'
Took 8.4302 seconds
HBaseTest@267498 get 'crime_records', '099#TEST#999999999'
COLUMN CELL
incident:case_number timestamp=2026-18-82718:51:02.032, value=TEST1234
1 row(s)
Took 8.9653 seconds
HBaseTest@267498 deleteall 'crime_records', '099#TEST#999999999'
Took 8.9116 seconds
HBaseTest@267498 get 'crime_records', '099#TEST#999999999'
COLUMN CELL
0 row(s)
Took 8.9116 seconds
HBaseTest@267498
```

Data Modification & Deletion:

- ▶ **Record Insertion (put):** Manually writes test record 099#TEST#999999999 across incident and details.
- ▶ **Column Delete (delete):** Drops specific cell details:isbeat; verified by get showing only remaining fields.
- ▶ **Row Purge (deleteall):** Writes tombstones to expunge the entire row; final get confirms **0 row(s)** remaining.
- ▶ **Lifecycle Verified:** Proves cell-level and row-level ACID mutations and consistency in HBase.

Step 5: HBase Java API Implementation & Live Execution

HBaseCrimeOperations.java:

```
Configuration config = HBaseConfiguration.create();
try (Connection conn = ConnectionFactory.createConnection(config);
    Table table = conn.getTable(TableName.valueOf("crime_records"))) {

    // Filtered Scan: Arrest tracking
    Scan scan = new Scan();
    SingleColumnValueFilter filter = new SingleColumnValueFilter(
        Bytes.toBytes("details"), Bytes.toBytes("arrest"),
        CompareOperator.EQUAL, new BinaryComparator(Bytes.toBytes("true")))
    );
    filter.setFilterIfMissing(true);
    scan.setFilter(filter).setLimit(5);

    try (ResultScanner scanner = table.getScanner(scan)) {
        for (Result r : scanner) {
            System.out.println("Match: " + Bytes.toString(r.getRow()));
        }
    }
}
```

Pipeline: Admin check → Put → Get → Filter Scan → Delete & Verify.

Live Execution Proof (run_java_api.sh):

[illegible]

- ▶ **Programmatic API:** Connects via `ConnectionFactory` executing the full CRUD pipeline in Java.
- ▶ **Filtered Scanner:** Programmatically tracks arrests and streams row keys directly from HBase.
- ▶ **Live Verification:** Terminal log confirms **[SUCCESS]** with zero connection or RPC errors.

Conclusion & Evaluation Rubric Compliance

Project Review 2 Summary:

- ▶ Successfully ported crime analytics from batch OLAP (Review 1) to real-time NoSQL serving on **Apache HBase**.
- ▶ Balanced load across RegionServers using composite key (District#CrimeType#ID).
- ▶ Demonstrated comprehensive HBase Shell commands (describe, get, filters, count, delete) with sub-second response times.
- ▶ Implemented native Java API client (HBaseCrimeOperations.java) executing full lifecycle operations.
- ▶ Verified on 10,000 authentic City of Chicago crime records.

Evaluation Criteria	Marks	Evidence
Real-world problem & dataset selection	1	Chicago Crimes authentic dataset (10K+ records, 22 attributes)
HBase table design & schema	1	Composite row key (Dist#Type#ID); 3 column families
HBase Shell implementation	2	create, describe, put, get, scan, count, delete, deleteall verified
HBase Filters & analytical queries	4	$\geq$ 3 filters: SingleColumnValueFilter, PrefixFilter, ValueFilter
Java API implementation	2	HBaseCrimeOps.java: Admin, Put, Get, Filter Scan, Delete
All rubric criteria fully satisfied		

Thank You! Questions & Live Demonstration.